진행 보고: 클래스별 넘김 임계값은 새 씨앗에서 재현되지 않았고, 논문 초안을 마쳤습니다

학부 졸업논문 · 보고일 2026-10-31 · 직전 보고(2026-10-27) 이후분

1. 요약

직전 보고의 후보 A(문자 단위 CNN → DistilBERT)를 중심으로 확인 실험 다섯 가지를 했고, 모두 판정 기준을 측정 전에 저장소에 적어 두었습니다. 그 결과를 반영해 정보보호학회논문지 투고 양식으로 논문 초안을 완성했습니다.

- **원 논문 설정(128 토큰) 재구현:** DistilBERT 를 128 토큰으로 학습하면 512 토큰보다 Macro-F1 이 1.96·1.70%p 낮고, 문자 단위 CNN 단독보다도 낮았습니다. 손실은 잘린 요청에 몰려 있고 SQLi 를 CmdI 로 착각하는 오답이 크게 늘었습니다(4절).
- **앞단을 작은 출구로 바꾼 캐스케이드:** 앞단을 후보 C 의 함께 학습한 작은 출구로 바꾸면 후보 A 보다 2.3~2.4배 빨라지고 정확도는 0.3%p 이내였습니다(두 씨앗 모두 기준 충족). 다만 DistilBERT 단독과 비교하면 0.31·0.28%p 낮습니다(5절).
- **클래스별 넘김 임계값은 새 씨앗 44·45 에서 재현되지 않았습니다.** 한 씨앗에서는 오히려 21.6% 느려졌고, 두 씨앗 모두 유의하지 않았습니다. 넘김 판단 쪽에서 "개선"이라고 쓸 결과는 이번에도 없습니다(6·7절).
- **변형 공격:** 공격 요청을 URL 인코딩 한 번만 해도 공격 요청당 기대 지연이 6.4배 늘었습니다. 더 큰 문제는 앞단이 공격으로 판정하는 비율이 99.4% 에서 86.7% 로 떨어져 일부 공격이 그대로 통과한다는 점입니다(8절).
- 폭 3 단독 모델의 학습 안정화 대조, 다중 스레드 지연, 2인 라벨 감사는 하지 않기로 정했습니다(9절).

용어는 직전 보고와 같습니다. "속도 배수"는 비교 기준 모델의 요청당 지연을 비교 구성의 **평균 지연**으로 나눈 값(fold 배수의 조화평균)이고, 별도 표시가 없으면 실제 사용자 정상 요청 기준·공격 비율 3.3% 로 환산했습니다. 지연은 전처리를 포함한 CPU 스레드 1개·요청 하나씩의 중앙값입니다. "기본 넘김 규칙"은 확신도 하나의 임계값으로 넘기는 방식, "클래스별 임계값"은 앞단이 예측한 클래스마다 임계값을 따로 두는 방식입니다.

2. 연구 범위 정리

후보 A·C 의 결과로 좋은 구성이 모두 "텍스트 모델 → 텍스트 모델"이라는 점이 분명해져, 제안 구성을 앞단·뒷단 모두 딥러닝으로 좁혔습니다. 이미지 CNN 을 앞단에 두는 구성은 공정 지연 기준에서 속도 이득이 없었고(10-14 보고), 논문 초안에서는 이미지 변환 실험을 빼고 문자 단위 CNN 과 DistilBERT 의 캐스케이드에 집중했습니다. 고전 기계학습 모델은 비교 기준으로만 남겼습니다.

3. DistilBERT 는 무엇을 더 잘 잡는가 (사후 분석)

씨앗 43 실패 뒤, DistilBERT 가 문자 단위 CNN 보다 무엇을 더 맞히는지 같은 요청끼리 비교했습니다. 결과를 보고 고른 분석이라 판정에는 쓰지 않습니다.

오답 종류 (5 fold 합산)	씨앗 42: 문자 단위 CNN → DistilBERT	씨앗 43
차단 실패 (공격 → 정상)	791 → 736 (−7%)	839 → 777 (−7%)
정상 오탐 (정상 → 공격)	206 → 245 (+19%)	140 → 118 (−16%)
공격 종류 혼동 (차단은 됨)	266 → 167 (−37%)	359 → 59 (−84%)
ㄴ 그중 SQLi 를 CmdI 로 착각	192 → 99	252 → 2

DistilBERT 의 이득은 공격을 더 막는 데가 아니라 **공격 종류 구분**, 특히 SQLi 를 가장 작은 클래스인 CmdI 로 착각하는 오답을 줄이는 데서 나왔습니다. 차단 실패는 두 씨앗 모두 약 7% 줄었을 뿐입니다. Macro-F1 이 작은 클래스에 민감해서 전체 차이의 대부분이 이 혼동에서 나옵니다.

캐스케이드(기본 넘김 규칙)는 시험 요청의 0.7~0.8% 만 넘기면서 SQLi→CmdI 오답을 192 → 123건(씨앗 42), 252 → 113건(씨앗 43)으로 줄여, DistilBERT 단독이 줄인 몫의 74% · 56% 를 가져왔습니다. DistilBERT 가 고친 오답 대부분은 문자 단위 CNN 이 덜 확신한 요청이었기 때문입니다.

4. 원 논문 설정(128 토큰) 재구현

직전 보고의 DistilBERT 는 잘림을 없애려 512 토큰을 썼지만, 유사 연구는 128 토큰을 썼습니다. 128 토큰에서는 요청의 6.9%, CmdI 의 26.5% 가 잘립니다. 512 판본을 주 기준으로 두고, 원 설정을 따로 학습해 나란히 보고하기로 측정 전에 정했습니다.

항목	씨앗 42: 512 → 128	씨앗 43: 512 → 128
Macro-F1	0.9914 → 0.9718 (−1.96%p, p 0.006)	0.9947 → 0.9777 (−1.70%p, p 0.006)
CmdI F1	0.9784 → 0.9038	0.9903 → 0.9284
SQLi 를 CmdI 로 착각	99 → 847	2 → 551
잘린 요청(15,553건) 정답률	0.994 → 0.957	0.996 → 0.956
안 잘린 요청 정답률	0.995 → 0.994	0.996 → 0.995

p 는 fold 간 학습 데이터 중복을 보정한 t-검정. 비교로 문자 단위 CNN 단독은 0.9884 · 0.9862 입니다.

- 미리 정한 기준(두 씨앗 모두 낮고 씨앗 42 가 유의)을 채워 **"원 설정(128 토큰)은 512 토큰보다 낮다"**로 기록했습니다. 128 토큰 DistilBERT 는 이 데이터에서 문자 단위 CNN 단독보다도 낮습니다.
- 손실은 잘린 요청에 몰려 있습니다. 잘린 요청의 78% 가 SQLi 라서, 뒷부분이 잘린 SQLi 를 CmdI 로 착각하는 오답이 크게 늘었습니다.
- 요청 하나씩 처리할 때는 대부분(93%)의 요청이 원래 128 토큰 이하라, 128 로 잘라도 지연이 거의 줄지 않았습니다(46.10 → 46.08ms).
- 원 설정끼리 캐스케이드(문자 단위 CNN → 128 토큰 DistilBERT)를 만들면 128 토큰 DistilBERT 단독보다 67.5배·76.3배 빠르고 더 정확해 기준상 "위"입니다. 그러나 넘김이 사실상 0% 라 캐스케이드가 한 일은 없고, "문자 단위 CNN 단독이 128 토큰 DistilBERT 보다 정확하고 76배 빠르다"를 다시 쓴 것에 불과합니다. 이 결과로 캐스케이드의 이득을 말하지 않겠습니다.

5. 앞단을 함께 학습한 작은 출구로 바꾼 캐스케이드

후보 A 에서는 넘김이 줄수록 비용의 대부분이 앞단(문자 단위 CNN 0.604ms)이 됩니다. 그래서 앞단을 후보 C 에서 함께 학습한 폭 3 출구(0.159ms)로 바꾸고 뒷단은 같은 DistilBERT 로 두었습니다. 새 학습 없이 기존 모델 출력으로 계산했습니다.

씨앗	속도 배수 (DistilBERT 단독 기준): 후보 A → 작은 출구 앞단	후보 A 대비 Macro-F1	DistilBERT 단독 대비 Macro-F1	판정 (후보 A 대비)
42	12.2 → 29.3배 (×2.40)	−0.28%p	−0.31%p	위
43	10.5 → 23.8배 (×2.27)	+0.05%p	−0.28%p	위

기준: 후보 A 보다 1.2배 이상 빠르고 Macro-F1 이 0.3%p 넘게 낮지 않을 것. 이번 실행의 지연이라 후보 A 의 배수가 직전 보고(12.3·10.6배)와 조금 다릅니다.

두 씨앗 모두 미리 정한 기준을 채워 **"후보 A 보다 위(2/2)"**로 기록했습니다. 단 세 가지를 함께 밝혀 둡니다. 씨앗 42 의 −0.28%p 는 허용폭 0.3%p 의 경계입니다. 이득의 상당 부분은 넘김이 아니라 앞단 자체가 4배 싸진 데서 옵니다(넘김 0% 인 fold 가 있습니다). DistilBERT 단독과 비교하면 씨앗 42 에서 허용폭을 0.01%p 넘으므로 "DistilBERT 보다 위"라고는 쓰지 않습니다.

6. 기본 넘김 규칙의 느린 fold 진단 (사후 분석)

직전 보고에서 씨앗 42 의 후보 A 는 fold 4 만 2.95배였습니다(나머지 46.9~80.2배). 원인을 fold 별로 분해했습니다.

fold	검증에서 고른 넘김 예산 → 임계값	속도 배수	실제 사용자 정상 넘김 (95% 구간)
1 · 2 · 3 · 5	0~1.0% → 0.37~0.87	46.9~80.2	0~0.9%
4	**3.0% → 0.985**	**2.95**	**33.6% [25.4, 43.0]**

fold 4 의 문자 단위 CNN 은 검증 Macro-F1 목표(DistilBERT 대비 −0.5%p)에 닿으려면 임계값을 0.985 까지 올려야 했고, 그러자 실제 사용자 정상 요청의 약 1/3 이 임계값 아래로 내려가 넘어갔습니다. 넘긴 36건 중 34건은 앞단이 이미 **정상으로 맞게** 예측한 요청이었습니다. 검증 데이터에서도 같은 비율(28.9%)이 보여 우연이 아니라 규칙의 결과입니다. 즉 임계값 하나로는 "공격 쪽 정확도를 맞추려 임계값을 올리면 덜 확신한 정상 요청이 대량으로 넘어간다"는 구조적 비용이 있습니다. 이것이 7절의 확인 실험을 한 이유입니다.

7. 클래스별 넘김 임계값 확인 (새 씨앗 44·45)

직전 보고에서 클래스별 임계값은 씨앗 42·43 에서 51.1배·42.5배로 기본 규칙보다 훨씬 빨랐지만, 둘 다 이미 본 씨앗의 사후 관찰이었습니다. 그래서 측정 전에 기준을 적고 아직 보지 않은 씨앗 44·45 로 DistilBERT 를 다시 학습해 확인했습니다. 기준은 씨앗마다 ① 속도 배수 20% 이상 증가 ② Macro-F1 0.3%p 넘게 낮지 않음 ③ fold 별 기대 넘김률 차이가 유의($p < 0.05$)입니다.

씨앗	속도 배수: 기본 → 클래스별	① 증가율	② Macro-F1	③ p	결과
44	76.0 → 59.6배	**−21.6%** ✗	0.9908 → 0.9920 ✓	0.34 ✗	①③ 미충족
45	42.4 → 54.4배	+28.4% ✓	0.9903 → 0.9913 ✓	0.54 ✗	③ 미충족

두 씨앗 모두 세 조건을 채우지 못해, **"씨앗 42·43 에서 본 클래스별 임계값의 큰 이득은 새 씨앗에서 재현되지 않았다"**로 기록했습니다. 이득은 기본 규칙이 6절과 같은 나쁜 운영점을 고른 fold 에서만 생겼습니다. 씨앗 45 의 fold 1 은 17.6 → 32.5배로 메워졌지만, 씨앗 44 는 기본 규칙이 이미 다섯 fold 모두 상한(76.3배)에 붙어 있어 줄일 넘김이 없었고 오히려 두 fold 를 깎았습니다. 즉 클래스별 임계값은 평균 비용을 줄이는 방법이 아니라 **나쁜 fold 를 일부 메우고 때로 좋은 fold 를 깎는** 방법입니다. 넘김 점수와 임계값을 바꾼 이전 시도들(10-04·10-14 보고)과 같은 방향이며, 넘김 판단의 한계를 딥러닝 구성에서 한 번 더 확인한 결과입니다.

병기(판정 아님): 기본 규칙 후보 A 와 같은 씨앗의 DistilBERT 단독을 비교하면 씨앗 44 +0.08%p, 씨앗 45 −0.24%p 로 둘 다 직전 보고의 세 조건을 채웠습니다. 네 씨앗(42~45)을 나란히 놓으면 3개에서 성립하고 씨앗 43 만 −0.32%p 로 못 채웠습니다. 다만 사전 확인 씨앗은 43 이었으므로 "씨앗 42 한정"이라는 직전 판정은 그대로 둡니다.

8. 넘김을 늘리는 변형 공격의 비용

후보 A 에서는 넘긴 요청 하나가 문자 단위 CNN 약 76회 분량이라, 공격자가 앞단의 확신을 떨어뜨리면 비용이 크게 늘 수 있습니다. 모델에 접근하지 않는 고정 변형을 공격 요청에 걸어 넘김률과 기대 지연을 쟀습니다(씨앗 42, 새 학습은 앞단 가중치 저장용 재학습뿐).

변형	공격 넘김률 (기본 규칙)	공격 요청당 기대 지연	공격 넘김률 (클래스별)	앞단 공격 판정률
URL 인코딩	0.27% → **8.76%** (×31.9)	0.73 → **4.64ms** (×6.4)	0.28% → 1.95% (×6.9)	99.4% → **86.7%**
이중 URL 인코딩	0.27% → 2.10% (×7.7)	0.73 → 1.57ms	0.28% → 0.44%	99.4% → 99.97%
대소문자 섞기	0.27% → 0.87% (×3.2)	0.73 → 1.00ms	0.28% → 0.50%	99.4% → 99.5%

- **비용 공격은 성립합니다.** 적응형 추론을 느리게 만드는 기존 공격(DeepSloth, ICLR 2021)이 웹 공격 탐지 캐스케이드에서도 페이로드 변형만으로 옮겨 옵니다. 이미지 CNN 앞단에서 넘김 ×1.8 이던 것이 여기서는 앞뒤 지연 격차 때문에 비용으로 크게 증폭됐습니다.
- **상한은 있습니다.** 변형된 공격도 4.64ms 로 DistilBERT 단독(46.1ms)의 약 1/10 이고, 공격 비율 3.3% 배포에서 평균 비용 증가는 약 0.13ms 입니다.
- 클래스별 임계값은 변형에 따른 넘김 증가를 기본 규칙보다 덜 겪었습니다(평균 배율 비 0.25). 다만 7절에서 비용 이득이 재현되지 않았으므로 이것을 방어책으로 내세우지는 않겠습니다.
- **더 심각한 것은 탐지 회피입니다.** URL 인코딩은 앞단의 공격 판정률을 12.7%p 떨어뜨렸습니다. 앞단이 정상으로 확신한 변형 공격은 뒷단으로 넘어가지 않으므로 그대로 통과합니다. 뒷단까지 포함한 최종 정확도는 fold 별 DistilBERT 가중치를 저장하지 않아 재지 못했고, 논문에는 한계로 적었습니다.

9. 하지 않기로 정한 것

- **폭 3 단독 모델의 학습 안정화 대조**(10-25 보고의 다음 계획 1): 결과가 어느 쪽이든 후보 C 를 "2차 모델 압축"으로 쓰는 문장이 바뀌지 않아, 측정 전에 뺐습니다. 논문에는 "함께 학습한 효과인지 학습 안정성 차이인지 가리지 못했다"고 적었습니다.
- **CPU 다중 스레드(4·8) 지연:** 지연은 CPU 스레드 1개·요청 하나씩만 보고하고, 다른 배포 조건은 한계로 적었습니다. 다중 스레드나 GPU 배치에서는 트랜스포머의 병렬화 효율 때문에 배수가 작아질 수 있습니다.
- **2인 라벨 감사:** 앞단이 확신하며 틀린 요청의 70.9% 가 라벨 불일치로 보인다는 관찰은 대형 언어모델 1인 판정이라, 논문 한계 절에 "가능성" 수준으로만 적었습니다. 성능 수치는 원 라벨 기준이라 영향이 없습니다.

10. 논문 초안

정보보호학회논문지 투고 양식(심사용, 저자·소속 없음)에 맞춰 초안을 완성했습니다.

- **제목:** 문자 수준 CNN과 DistilBERT 기반 캐스케이드 웹 공격 탐지의 속도–정확도 교환 분석
- **구성:** 서론 · 관련 연구 · 방법 · 실험 환경 · 실험 결과(딥러닝 구성 네 가지의 속도–정확도, 넘김 판단의 한계, CmdI 오답 분해와 128 토큰 재구현) · 논의 및 한계 · 결론. 표 3개, 그림 1개, 참고문헌 21편.
- **주장 강도:** 후보 A 의 "DistilBERT 단독보다 위"는 씨앗 42 한정으로, 후보 C 는 넘김이 아니라 2차 모델 압축으로, 클래스별 임계값은 재현 실패로 적었습니다. 3절의 오답 분해는 사후 분석으로 표시했고, "개선"이라는 표현은 쓰지 않았습니다.
- **결론 문장:** 캐스케이드의 성패는 넘김 임계값 조정이 아니라 앞뒤 지연 격차와 앞단 모델의 판별력·변형 강건성에 달려 있으며, 성능 주장은 공정 지연·배포 공격 비율·실제 사용자 정상 넘김률·씨앗 간 분산을 함께 보고할 때만 성립한다.

11. 다음에 할 것

1. 참고문헌 서지 정보를 원문과 대조합니다.

2. 초안을 학회 HWP 양식(2단)으로 옮기고 수식을 다시 입력합니다.

참고: 재현 정보

측정 기기: AMD Ryzen 7 9800X3D(8코어 16스레드), RAM 32GB, NVIDIA RTX 4080 SUPER, Windows 11, PyTorch 2.6.0(CUDA 12.4). 보고 기간 2026-10-27~10-31. 4·5·7·8절의 지연은 2026-10-31 한 실행에서 번갈아 잰 값(문자 단위 CNN 0.604ms · 폭 3 출구 0.159ms · DistilBERT 512 토큰 46.10ms · 128 토큰 46.08ms, 검증 2,000건 × 30회 중앙값)이고, 직전 보고의 실행 값(0.563 · 45.15ms)과 섞지 않았습니다. 판정 기준은 측정 전에 커밋했습니다(4절 b341e9f · 5·7·8절 d9d3678). 분석 코드는 fc9f513 · bfff884, 하지 않기로 한 결정은 측정 전 7ed10d2 입니다. 8절의 앞단은 가중치 저장을 위해 씨앗 42 로 다시 학습한 모델이라 운영점이 원 실행과 조금 다릅니다(같은 실행 값끼리만 비교).